

R Programming in SAS® LSAF: How to Generate Clinical Reports Using an R Session

Praneeth Adidela, ICON plc

ABSTRACT

The SAS® Life Science Analytics Framework (LSAF) provides an integrated system for transforming, analyzing, reporting, and reviewing clinical research data. LSAF is a cloud-native, single solution for clinical analysis and submission and offers integrated features for clinical data repository, regulatory compliance, quality control, job manifests, version control, audit trails, e-signatures, and documentation support. It enables automation through workflows, supports integrations, and ensures the proper implementation and management of data standards and controlled terminology.

LSAF version 5.3 and later introduces integration with R. The program editor within LSAF allows users to code in SAS or R on the same flexible, open platform. Users can develop R programs in LSAF within the R session using an interface similar to the SAS session. The repository and workspace store R program files (*.r), data files (*.rdata or *.rds), and log files (*.rlog) like their SAS counterparts. Users can add R programs to jobs and incorporate SAS and R programs within the same job. The processes for running and debugging R programs are similar to those for SAS programs. Compliance, performance, and traceability are the key benefits of using R in LSAF.

This paper will focus on utilizing R within LSAF, exploring the coding process on the platform to generate clinical reports, and offering strategies for working efficiently within LSAF.

INTRODUCTION

SAS® Life Science Analytics Framework (LSAF) is a cloud-native solution that provides an integrated system for managing, analyzing, reporting, and reviewing clinical research data. It offers a streamlined approach to clinical analysis and submission processes.

The hosted LSAF provides a structured platform for managing clinical data, consisting of a global repository and individual user workspace. The repository is a shared location with specific permissions and privileges to control access. It contains folders and files that store the organization's clinical assets, such as documents, programs, studies, and data standards. In contrast, the workspace in LSAF is a private and dynamic environment designed for individual users. Each user has a dedicated workspace with exclusive control to create, view, and modify files. To edit or modify a file in the repository, check out the file from the repository to the workspace. After completing the necessary updates or new developments, files can be checked back into the repository, making them accessible to others.

The system's structured job execution model requires users to create jobs to run programs. This design ensures reproducibility, consistency, and regulatory compliance in clinical data analytics. After a job runs, it generates a manifest file containing critical details. This file includes information such as the user who initiated the job, the method of execution, timestamps, logs, input and output files, and other essential metadata.

Version control mechanisms in LSAF ensure that all file modifications are systematically tracked and assigned unique version numbers, enabling users to manage, review, and revert to previous versions when necessary. A detailed audit trail also logs all significant actions, including job executions and file modifications. It captures essential metadata like the responsible user, the action performed, and the timestamp, thereby maintaining operational integrity and compliance. To further strengthen regulatory adherence, LSAF supports electronic signatures, allowing users to formally approve files within the repository while maintaining a detailed audit record, with role-based authentication ensuring secure identity verification. LSAF ensures regulatory compliance by following industry standards, including 21 CFR Part 11 and good practice guidelines (GxP), thus supporting secure and compliant clinical research and development.

LSAF offers automation through workflows and supports integrations with various databases, web interfaces, and third-party applications via the SAS/ACCESS engine, REST APIs, and Java/SAS APIs. Standard integrations include Veeva® CTMS and CDMS, Pinnacle 21® (Community and Enterprise), CDISC library, Smartsheet, and Medidata Rave® EDC, ensuring seamless end-to-end clinical data management.

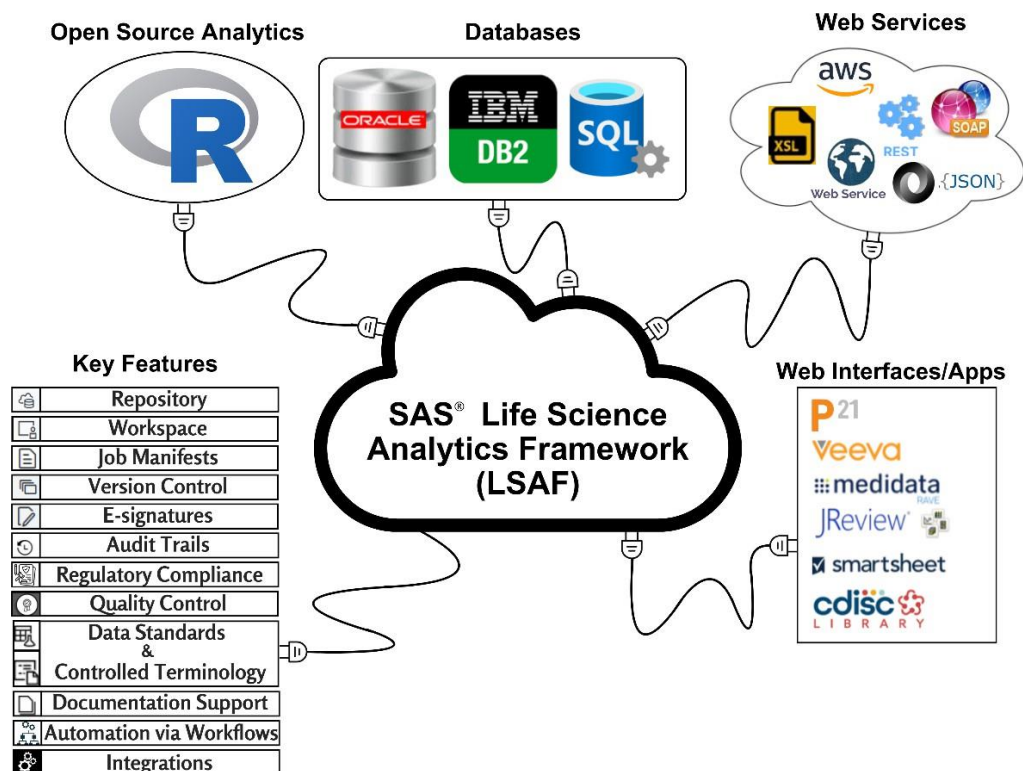


Figure 1. Overview of SAS LSAF

R IN LSAF

LSAF version 5.3 and later introduces integration with R, allowing users to code in SAS or R within the same flexible and open platform. The program editor in LSAF enables users to develop R programs within an R session using an interface similar to the SAS session. The system manages R sessions in the same manner as SAS sessions. LSAF version 5.4 includes enhancements to Data Explorer that support data frames. This enhancement includes features for viewing the contents of RData files, exploring the metadata and content of R-formatted data frames from RData and RDS files, and filtering and sorting these data frames. Users can also save a subset of a data frame to the workspace within RData and RDS files. The repository and workspace can store R program files (*.r), data files (*.rdata, *.rds), and log files (*.rlog), similar to their SAS equivalents. Users can incorporate R programs into jobs, including both SAS and R programs within the same job. Running and debugging R programs follows the same process as SAS programs.

Several variables are automatically created when an R program is executed, and users can reference these variables within the R program:

- **LSAF_USR** This specifies the user ID of the individual running the R program.
- **LSAF_WS** This indicates the location of the root-level folder for the workspace.
- **LSAF_USRWS** This denotes the path to the user's specific folder within the workspace.
- **LSAF_INSTANCE** This specifies the product instance name.

- **LSAF_FILELOCATION** This refers to the location of the R program that was most recently submitted during the R session; it does not include the program's file name.
- **LSAF_FILEPATH** This indicates the path of the R program that was most recently submitted during the R session, including the program file name.

```
> # Global Session Variables
> LSAF_USR <- "padidela"
> LSAF_WS <- "/lsafshared/SASWorkspaces/padidela"
> LSAF_USRWS <- "/lsafshared/SASWorkspaces/padidela/Users/padidela"
> LSAF_INSTANCE <- "xahprod.ondemand.sas.com"
> LSAF_FILELOCATION <- "/R/library"
> LSAF_FILEPATH <- "/R/library/f_1_blalt_ab.R*"
```

EXAMPLES OF CREATING TLFs

The examples below illustrate the use of R in LSAF for generating TLFs.

EXAMPLE 1: DEMOGRAPHIC TABLE

The first example demonstrates the step-by-step process of building the R script to generate a typical demographic table. It uses ADSL SAS dataset as the analysis dataframe to produce demographic results.

The demographic table is created using the pre-installed R packages in LSAF, including haven, dplyr, and additional packages such as Tplyr and pharmaRTF, which are sourced from the local library in the LSAF workspace. The haven package allows for the reading and writing of data in SPSS, Stata, and SAS formats. The dplyr package offers a powerful and intuitive framework for data manipulation, providing consistent syntax for filtering, selecting, and transforming data frames. It is often referred to as a “grammar of data manipulation.” Tplyr is a traceability-focused tool designed to simplify data manipulation for generating clinical summaries. The pharmaRTF package is an advanced RTF wrapper that facilitates the creation of reports, particularly for regulatory submissions.

```
# Load necessary libraries
library(Tplyr)
library(pharmaRTF)
library(haven)
library(dplyr)

# data
filepath <- file.path(LSAF_WS, "/R/Study/Data/adsl.sas7bdat")
data <- read_sas(filepath)

# Select required variables
adsl <- data %>% dplyr::select(USUBJID, AGE, SEX, ETHNICITY, RACE, HEIGHT, WEIGHT, BMI, TRT01A)

# Create the Tplyr Table
t <- tplyr_table(adsl, TRT01A) %>%

# Create a total group column
add_total_group() %>%

# Add descriptive statistics layer for AGE
add_layer(name = 'AGE',
group_desc(AGE, by = "Age (Years)") %>%
set_format_strings(
  "n" = f_str("xx", n),
  "Mean (SD)" = f_str("xx.x (xx.xx)", mean, sd),
  "Median" = f_str("xx.x", median),
  "Q1, Q3" = f_str("xx.x, xx.x", q1, q3),
  "Min, Max" = f_str("xx, xx", min, max))) %>%

# Add count layer for SEX
add_layer(name = 'SEX',
group_count(SEX, by = "Sex n (%)") %>%
```

```

# Add count layer for RACE
add_layer(name = 'RACE',
group_count(RACE, by = "Race n (%)")) %>%
# Add count layer for ETHNICITY
add_layer(name = 'ETHNICITY',
group_count(ETHNICITY, by = "Ethnicity n (%)")) %>%
# Add count layer for HEIGHT
add_layer(name = 'HEIGHT',
group_desc(HEIGHT, by = "Height (cm)") %>%
set_format_strings( "n" = f_str("xx", n),
                    "Mean (SD)" = f_str("xx.x (xx.xx)", mean, sd),
                    "Median" = f_str("xx.x", median),
                    "Q1, Q3" = f_str("xx.x, xx.x", q1, q3),
                    "Min, Max" = f_str("xx.x, xx.x", min, max))) %>%
# Add count layer for WEIGHT
add_layer(name = 'WEIGHT',
group_desc(WEIGHT, by = "Weight (kg)") %>%
set_format_strings( "n" = f_str("xx", n),
                    "Mean (SD)" = f_str("xx.x (xx.xx)", mean, sd),
                    "Median" = f_str("xx.x", median),
                    "Q1, Q3" = f_str("xx.x, xx.x", q1, q3),
                    "Min, Max" = f_str("xx.x, xx.x", min, max))) %>%
# Add count layer for BMI
add_layer(name = 'BMI',
group_desc(BMI, by = "BMI (kg/m2)") %>%
set_format_strings( "n" = f_str("xx", n),
                    "Mean (SD)" = f_str("xx.x (xx.xx)", mean, sd),
                    "Median" = f_str("xx.x", median),
                    "Q1, Q3" = f_str("xx.x, xx.x", q1, q3),
                    "Min, Max" = f_str("xx.x, xx.x", min, max)))

# Build the Tplyr table to get the data
dat <- build(t)

# Add column headers to the data frame
dat <- dat %>%
arrange(ord_layer_index, ord_layer_1, ord_layer_2) %>%
apply_row_masks(row_breaks = TRUE) %>%
select(starts_with("row_label"), `var1_Treatment A`, `var1_Treatment B`, `var1_Total`) %>%
add_column_headers(s = paste0(" | Treatment A\\line(N=**Treatment A**)
                                | Treatment B\\line(N=**Treatment B**) ",
                                "| Total\\line(N=**Total**)"),
header_n = header_n(t))

# Make the table
ht <- huxtable::as_hux(dat, add_colnames = FALSE) %>%
  huxtable::set_bold(1, 1:ncol(dat), TRUE) %>% # Bold the first row (header)
  huxtable::set_align(1, 1:ncol(dat), 'center') %>% # Center align header row
  huxtable::set_align(2:nrow(dat), 3:ncol(dat), 'center') %>% # Center align the results
  huxtable::set_top_border(1, 1:ncol(dat), 1) %>% # Add a top border to header row
  huxtable::set_bottom_border(1, 1:ncol(dat), 1) %>% # Add a bottom border to header row
  huxtable::set_valign(1:nrow(dat), 1:ncol(dat), 'bottom') %>% # Align content to bottom
  huxtable::set_width(1.5) %>% # Set the table width
  huxtable::set_escape_contents(FALSE) %>% # Don't escape RTF syntax
  huxtable::set_col_width(c(.15, .28, .15, .15, .15)) # Ensure correct number of columns

#pharmaRTF to create the RTF output
rtf <- rtf_doc(ht) %>%

# Add titles
add_titles(
hf_line("Protocol: XXXXXXXX", "PAGE_FORMAT: Page %s of %s", align='split', bold=TRUE, italic=FALSE),
hf_line("Table 1. Demographic and Baseline Characteristics",
        align='center', bold=TRUE, italic=FALSE)) %>%

# Add footnotes
add_footnotes(
hf_line("Study: XXXXXXXX, Program: t_1_demog.R, Output: t_1_demog.rtf",
align='left', bold=FALSE, italic=FALSE)) %>%
set_font_size(10) %>%
set_footer_height(1) %>%

```

```
# remove extra padding
set_ignore_cell_padding(TRUE)

#write out the RTF file!
write_rtf(rtf, file='t_1_demog.rtf')
```

Protocol: XXXXXXXX

Page 1 of 1

Table 1. Demographic and Baseline Characteristics

		Treatment A (N=55)	Treatment B (N=46)	Total (N=101)
Age (Years)	n	55	46	101
	Mean (SD)	69.3 (15.84)	71.0 (16.30)	70.1 (15.99)
	Median	68.0	71.5	69.0
	Q1, Q3	60.0, 79.0	62.2, 83.2	60.0, 81.0
	Min, Max	29, 99	37, 99	29, 99
Sex n (%)	Female	23 (41.8%)	15 (32.6%)	38 (37.6%)
	Male	32 (58.2%)	31 (67.4%)	63 (62.4%)
Race n (%)	American Indian/Alaska Native	2 (3.6%)	4 (8.7%)	6 (5.9%)
	Asian	35 (63.6%)	24 (52.2%)	59 (58.4%)
	Black/African American	4 (7.3%)	6 (13.0%)	10 (9.9%)
	Native Hawaiian/Other Pacific Islander	7 (12.7%)	4 (8.7%)	11 (10.9%)
	Other	7 (12.7%)	8 (17.4%)	15 (14.9%)
Ethnicity n (%)	Hispanic or Latino	2 (3.6%)	9 (19.6%)	11 (10.9%)
	Not Hispanic or Latino	53 (96.4%)	37 (80.4%)	90 (89.1%)
Height (cm)	n	55	46	101
	Mean (SD)	1.6 (0.09)	1.6 (0.09)	1.6 (0.09)
	Median	1.6	1.6	1.6
	Q1, Q3	1.6, 1.7	1.6, 1.7	1.6, 1.7
	Min, Max	1.5, 1.8	1.5, 1.8	1.5, 1.8
Weight (kg)	n	55	46	101
	Mean (SD)	62.5 (13.85)	61.5 (12.62)	62.1 (13.25)
	Median	59.6	61.5	60.0
	Q1, Q3	52.8, 69.0	52.2, 68.0	52.6, 69.0
	Min, Max	42.0, 106.0	38.0, 107.0	38.0, 107.0
BMI (kg/m2)	n	55	46	101
	Mean (SD)	23.1 (3.96)	22.9 (3.48)	23.0 (3.73)
	Median	23.1	22.7	22.9
	Q1, Q3	20.9, 24.3	20.9, 25.0	20.9, 24.4
	Min, Max	16.4, 35.0	16.0, 33.8	16.0, 35.0

Study: XXXXXXXX, Program: t_1_demog.R, Output: t_1_demog.rtf

EXAMPLE 2: LISTING OF ADVERSE EVENTS

The second example showcases the process of creating an adverse events listing using R. It utilizes ADAE SAS dataset as the analysis dataframe to generate the listing.

The AE listing is generated using the pre-installed haven R package in LSAF, along with additional packages such as tidyverse and r2rtf, accessed from the local library within the LSAF workspace. The tidyverse is a suite of R packages designed to simplify data science tasks, providing tools for data import, manipulation, and visualization with a consistent syntax and API. r2rtf is an R package designed to generate production-ready tables and figures in RTF format.

```
# Load necessary libraries

library(haven)
library(tidyverse)
library(r2rtf)
```

```

# Read SAS dataset
filepath <- file.path(LSAF_WS, "/R/Study/Data/adae.sas7bdat")
ADAE <- read_sas(filepath)
# Select necessary columns and convert data types

dt1 <- ADAE %>% select(USUBJID, AETERM, AESTDTC, AEENDTC, AEONGO, AESEV, AESER, AEREL, AEOUT) %>%
  mutate(
    USUBJID = as.character(USUBJID), # Convert numeric to character
    AESTDTC = format(AESTDTC, "%Y-%m-%d"), # Convert date to character
    AEENDTC = format(AEENDTC, "%Y-%m-%d") # Convert date to character
  ) %>%
  arrange(USUBJID) # Sort by subject

#foot_string

dt_time <- format(Sys.time(), "%Y-%m-%d %H:%M")
foot_string <- paste0("Study: xxxxx, Program: l_1_ae.R, Output: l_1_ae.rtf, ", dt_time) #

Generate RTF Report

dt1 %>%
rtf_page(col_width = 8, border_first = "single", border_last = "single") %>%

# Add Title

rtf_title(title = "Listing 1. Adverse Events", text_justification = c("l"), text_format="b") %>%

# Column Header

rtf_colheader(colheader = "Subject ID | AE Reported Term | AE start Date | AE End Date | Ongoing|
                          Severity | Serious Event | Causality | Outcome",
  col_rel_width = c(1.5, 2, 2, 2, 2, 2, 2, 3, 3.2),
  text_justification = c("l", "l", "c", "c", "c", "c", "c", "c", "c"),
  border_top = rep("", 9),
  border_right = rep("", 9),
  border_left = rep("", 9),
  text_format="b") %>%

# Body of the Table

rtf_body(col_rel_width = c(1.5, 2, 2, 2, 2, 2, 2, 3, 3.2),
  border_first=rep("single", 9),
  border_last=rep("single", 9),
  border_left=rep("", 9),
  border_right=rep("", 9),
  text_justification = c("l", "l", "c", "c", "c", "c", "c", "c", "l"),
  last_row=F) %>%

# Footnote

rtf_footnote(footnote = c( foot_string),
  border_left="",
  border_right="",
  border_bottom="",
  border_top="",
  text_font_size = 9,
  text_font = 1,
  text_format = NULL,
  as_table = FALSE,
  text_convert = FALSE) %>%

# Encode RTF settings

rtf_encode() %>%

# Save to RTF File

write_rtf("l_1_ae.rtf")

```

Listing 1. Adverse Events

Subject ID	AE Reported Term	AE start Date	AE End Date	Ongoing	Severity	Serious Event	Causality	Outcome
101	Arrhythmia	2025-02-13		Yes	Moderate	No	Not Related	Not recovered/Not resolved
102	Dizziness	2024-02-19	2024-02-20	No	Mild	No	Possibly Related	Recovered/Resolved
103	Headache	2024-02-21	2024-03-04	No	Moderate	No	Related	Recovered/Resolved
104	Infection	2025-02-17		Yes	Moderate	No	Unlikely Related	Not recovered/Not resolved
105	Constipation	2025-02-11		Yes	Moderate	No	Unlikely Related	Not recovered/Not resolved
106	Nausea	2024-02-19	2024-03-03	No	Mild	No	Related	Recovered/Resolved
107	Infection	2024-02-19	2024-03-03	No	Mild	No	Unlikely Related	Recovered/Resolved
108	Abdominal Pain	2024-03-03	2024-03-25	No	Severe	No	Possibly Related	Recovered/Resolved
109	Nausea	2025-03-17		Yes	Moderate	No	Related	Recovering/Resolving
110	Constipation	2024-03-03	2024-03-25	No	Mild	No	Unlikely Related	Recovered/Resolved

Study: xxxxx, Program: l_1_ae.R, Output: l_1_ae.rtf, 2025-05-22 14:05

EXAMPLE 3: BLAND-ALTMAN FIGURE

The third example clearly illustrates how to create an R script for generating a Bland-Altman plot. Raw data in excel is utilized to visually represent the Bland-Altman results.

The Bland-Altman plot is generated using ggplot2 and r2rtf. The r2rtf package is accessed from the local library within the LSAF workspace, along with the pre-installed ggplot2, readxl and dplyr R packages in LSAF. ggplot2 is an R package for creating statistical and data visualizations. Unlike most other graphics packages, it allows users to construct graphs by layering independent components, making it a powerful tool for data visualization.

```
# Load necessary libraries
library(readxl)
library(ggplot2)
library(dplyr)
library(r2rtf)

# data
filepath <- file.path(LSAF_WS, "R/Study/Data/raw.xlsx")
ef <- read_excel(filepath)

# Data preparation
ef <- ef %>% dplyr::select(SUBJECT, EDVCHCE, EDVCHGT)
#create new column for average measurement
ef$avg <- rowMeans(ef[, c("EDVCHCE", "EDVCHGT")])
#create new column for difference in measurements
ef$diff <- ef$EDVCHCE - ef$EDVCHGT
#find average difference
mean_diff <- mean(ef$diff)
#find standard deviation of the differences
sddiffRvol <- sd(ef$diff)
# Find lower and upper 95% confidence interval limits
lower <- mean_diff - 1.96*sd(ef$diff)
upper <- mean_diff + 1.96*sd(ef$diff)
#sample size (assuming n_ is the number of rows)
n_ <- nrow(ef)
#Calculate the t-value for 95% confidence interval, degrees of freedom = n_ - 1
t_value <- qt(0.975, df = n_ - 1)
#Calculate the lower and upper limits for LOA for lower and upper values
l1la <- lower - (t_value * sddiffRvol * sqrt(1/n_ + (1.96^2)/(2*(n_ - 1))))
ulla <- lower + (t_value * sddiffRvol * sqrt(1/n_ + (1.96^2)/(2*(n_ - 1))))
lula <- upper - (t_value * sddiffRvol * sqrt(1/n_ + (1.96^2)/(2*(n_ - 1))))
uula <- upper + (t_value * sddiffRvol * sqrt(1/n_ + (1.96^2)/(2*(n_ - 1))))
# Printing the calculated limits
cat("Lower interval of lower LOA (l1la):", l1la, "\n")
```

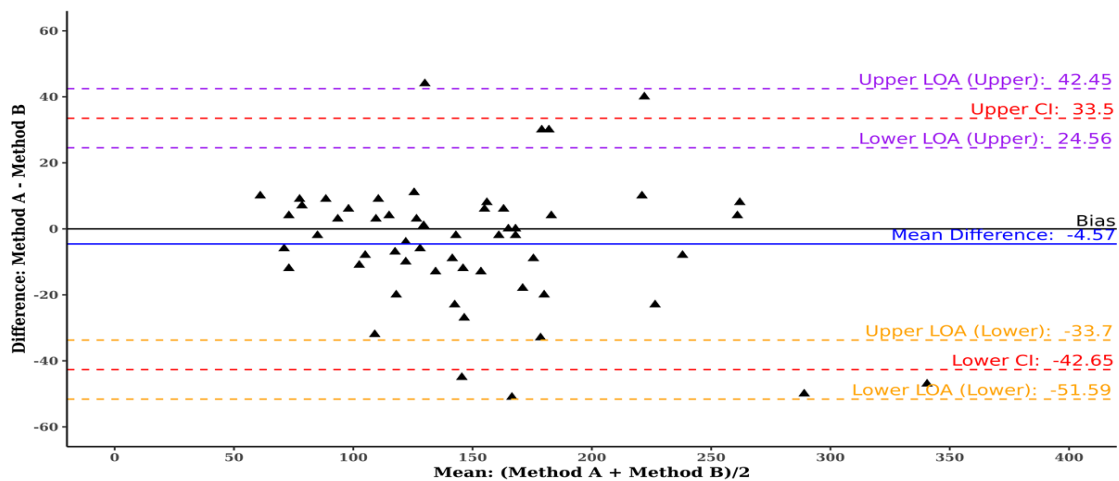


```

cat("Upper interval of lower LOA (ulla):", ulla, "\n")
cat("Lower interval of upper LOA (lula):", lula, "\n")
cat("Upper interval of upper LOA (uula):", uula, "\n")
# Calculate the range of the diff values
y_range <- diff(range(ef$diff))
# Create the Bland-Altman plot
final <- ggplot(ef, aes(x = avg, y = diff)) +
  geom_point(size = 2, shape = 17) +
  theme_classic()+
  geom_hline(yintercept = 0, color = "black") + # Bias line (middle line)
  geom_hline(yintercept = mean_diff, color = "blue") + # Mean difference line
  geom_hline(yintercept = lower, color = "red", linetype = "dashed") + # Lower CI line
  geom_hline(yintercept = upper, color = "red", linetype = "dashed") + # Upper CI line
  geom_hline(yintercept = llla, color = "orange", linetype = "dashed") + # Lower LOA (lower) line
  geom_hline(yintercept = ulla, color = "orange", linetype = "dashed") + # Upper LOA (lower) line
  geom_hline(yintercept = lula, color = "purple", linetype = "dashed") + # Lower LOA (upper) line
  geom_hline(yintercept = uula, color = "purple", linetype = "dashed") + # Upper LOA (upper) line
  theme(text = element_text(size = 10, family = "serif", face = "bold")) +
  labs(x="Mean: (Method A + Method B)/2",
       y="Difference: Method A - Method B") +
  # Set custom axis ranges
  scale_x_continuous(limits = c(0, 400), breaks = seq(0, 400, by = 50)) + # Adjust x-axis range
  scale_y_continuous(limits = c(-60, 60), breaks = seq(-60, 60, by = 20)) + # Adjust y-axis range
  theme(plot.caption = element_text(hjust = 0), #Default is hjust=1
        plot.title.position = "plot",
        plot.caption.position = "plot")+
  # Add annotations
  annotate("text", x = max(ef$avg), y = 0 + 2.5, label = "Bias",
          color = "black", hjust = -3.7) +
  annotate("text", x = max(ef$avg), y = mean_diff + 0.03 * y_range,
          label = paste("Mean Difference: ", round(mean_diff, 2)), color = "blue", hjust = 0.16) +
  annotate("text", x = max(ef$avg), y = lower + 0.03 * y_range,
          label = paste("Lower CI: ", round(lower, 2)), color = "red", hjust = -0.15) +
  annotate("text", x = max(ef$avg), y = upper + 0.03 * y_range,
          label = paste("Upper CI: ", round(upper, 2)), color = "red", hjust = -0.3) +
  annotate("text", x = max(ef$avg), y = llla + 0.03 * y_range,
          label = paste("Lower LOA (Lower): ", round(llla, 2)), color = "orange", hjust = 0.29) +
  annotate("text", x = max(ef$avg), y = ulla + 0.03 * y_range,
          label = paste("Upper LOA (Lower): ", round(ulla, 2)), color = "orange", hjust = 0.25) +
  annotate("text", x = max(ef$avg), y = lula + 0.03 * y_range,
          label = paste("Lower LOA (Upper): ", round(lula, 2)), color = "purple", hjust = 0.27) +
  annotate("text", x = max(ef$avg), y = uula + 0.03 * y_range,
          label = paste("Upper LOA (Upper): ", round(uula, 2)), color = "purple", hjust = 0.27)
# Save the plot
ggsave(filename = "f_1_blalt_ab.png", # output path as filename
        plot = final, # Specify the plot object
        device = "png", # Specify the image format (e.g., png, pdf,..)
        width = 9, # Set the plot width in inches
        height = 5, # Set the plot height in inches
        dpi = 2200) # Optional: Set the resolution (DPI) for higher quality
# Define the figure path
fig_path <- "f_1_blalt_ab.png"
# Read the figure from the specified path
fig_obj <- rtf_read_figure(fig_path)
# Generate RTF report with figure
fig_obj %>%
  rtf_page(orientation = "portrait")%>%
  # Add Title
  rtf_title("Figure. 1. Bland-Altman Plot of differences between two methods - A and B") %>%
  # Add Footnote
  rtf_footnote("Study: xxxxxx, Program: f_1_blalt_ab.R, Output: f_1_blalt_ab.rtf",
              text_convert = FALSE, text_justification="c") %>%
  # Adjust Height and Width
  rtf_figure(fig_height = 3,
            fig_width = 6) %>%
  # Encode the RTF file as a figure document
  rtf_encode(doc_type = "figure") %>%
  # Write the final RTF file
  write_rtf("f_1_blalt_ab.rtf")

```


Figure. 1. Bland-Altman Plot of differences between two methods - A and B



Study: xxxxxx, Program: f_1_blalt_ab.R, Output: f_1_blalt_ab.rtf

R - PACKAGE MANAGEMENT IN LSAF

The LSAF environment provides a base set of pre-installed R packages. Users can often add additional packages to their LSAF environment using a local library or a CRAN-like repository based on their specific analysis needs.

LOCAL LIBRARY

The `install.packages()` command is used to install R packages. By default, this command tries to save the installed package(s) in the global library, which usually requires administrative privileges. As a result, users may encounter permission issues when trying to install packages in this directory. Users can create a local library in their workspace to resolve this limitation and install packages there.

The `.libPaths()` command lists the locations where R searches for libraries. The first item on this list is the target for new package installations.

```
.libPaths()
```

```
> .libPaths()
[1] "/usr/lib64/R/library" "/usr/share/R/library"
```

Let's install the "r2rtf" package.

```
install.packages("r2rtf")
```

R attempts to install it in the global library but lacks permission to write in that location.

```
> install.packages("r2rtf")
Installing package into '/usr/lib64/R/library'
(as 'lib' is unspecified)
Warning in install.packages("r2rtf") :
  'lib = "/usr/lib64/R/library"' is not writable
Error in install.packages("r2rtf") : unable to install packages
```

Create a local library in the workspace.

```
R_LIBS_USER=~/.R/custom-lib/%v"
dir.create(Sys.getenv("R_LIBS_USER"), recursive = TRUE)
```

```
> .libPaths()
[1] "/lsafshared/SASWorkspaces/padidela/R/x86_64-redhat-linux-gnu-library/4.4"
[2] "/usr/lib64/R/library"
[3] "/usr/share/R/library"
```

Download the "r2rtf" package and dependencies onto the user's PC, upload them to the LSAF workspace, install the package in the local library, and call the package using the command 'library(r2rtf)'.

```
localLib <- paste0(LSAF_WS, "/R/x86_64-redhat-linux-gnu-library/4.4")
pkgPath <- paste0(LSAF_WS, "/R/Packages/r2rtf_1.1.4.tar.gz")
install.packages(pkgPath, lib=localLib, repos=NULL, type="source")
library("r2rtf")
```

LOCAL CRAN REPOSITORY

LSAF does not allow users to download or install packages directly from CRAN. Instead, users can create a local CRAN-like repository containing selected packages and upload it to LSAF. The miniCRAN package makes it possible to create a mini version of CRAN containing only selected packages. Users can then install packages from this mini CRAN repository.

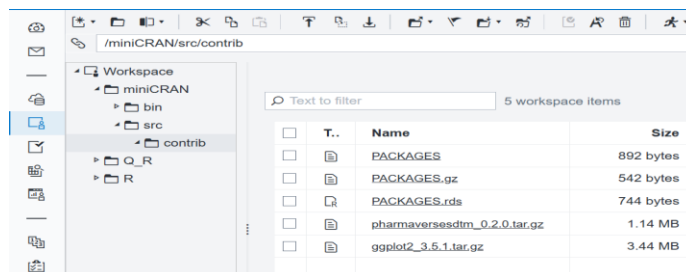
Use miniCRAN package within RStudio to create a local CRAN repository on the user's PC. The makeRepo() function in miniCRAN creates a repository.

```
#Install the 'miniCRAN' package
install.packages("miniCRAN")
# Load the 'miniCRAN'
library(miniCRAN)
# Step 1: Set CRAN mirror
cran_mirror <- "http://cran.rstudio.com"
# Step 2: Specify packages you want to include
pkgs <- c("pharmaversesdtm")
# Step 3: Create directory for local repo (change this path as needed)
local_repo <- file.path(tempdir(), "miniCRAN")
dir.create(local_repo, showWarnings = FALSE)
# Step 4: Get package dependencies recursively
pkg_deps <- pkgDep(pkgs, repos = cran_mirror, type = "source",
                   suggests = FALSE)
print(pkg_deps) # List of packages including dependencies
# Step 5: Download packages and dependencies to local repo
makeRepo(pkg_deps, path = local_repo, repos = cran_mirror, type =
c("source", "win.binary"))
```

Upload the local CRAN repository to LSAF and install packages from it into the R library.

The functions 'install.packages()' and 'uninstall.packages()' work effectively. Package dependencies are handled automatically.

```
localCRAN <- paste0("file:///", LSAF_WS, "/miniCRAN")
RLibrary <- paste0(LSAF_WS, "/R/x86_64-redhat-linux-gnu-library/4.4")
install.packages("pharmaversesdtm", repos=localCRAN, lib=RLibrary, type="source")
remove.packages("pharmaversesdtm ", lib = RLibrary)
```



SAS SESSION vs. R SESSION IN LSAF

The Sessions page in LSAF allows users to manage SAS and R sessions. Users can open multiple program editors in a single session editor, and each program editor shares the same session. Users can also open multiple session editors, each with a separate session; program editors in different session editors do not share the same session.

Feature	SAS Session	R Session
Code	Contains the program	Contains the program
Log	Contains the log file	Contains the log file
Results	Contains the results generated by the SAS program	Not available
Submission Inputs	Contains the input files that are used by the SAS program.	Not available
Submission Outputs	Contains the output files that are generated by the SAS program	Not available

USER EXPERIENCE

R IN LSAF 5.4 vs. RSTUDIO

R is widely used for statistical computing and can be executed in various environments. LSAF provides a controlled and validated R environment, ensuring compliance with regulatory standards and secure data access while integrating seamlessly with SAS. In contrast, RStudio offers a flexible integrated development environment (IDE), giving users complete control over script execution, package management, and system configurations.

The table below highlights the key differences between R in LSAF 5.4 and RStudio.

Feature	R in LSAF 5.4	RStudio
Environment	Embedded in LSAF, controlled and validated environment	Standalone R-focused application
User Interface	Limited to Code and Log windows only	Advanced UI with multiple panes (Console, Script, Environment, Plots, etc.)
Package Installation	Controlled Process: local library or miniCRAN allows additions.	Full access to CRAN, GitHub, Bioconductor, etc.
Package Dependencies	Iterative Process	Fully Automated
Traceability	Enabled through job manifests	Limited
Program Editor	Decent	Advanced
Log	Embedded	Manual via command line
Data Explorer	Data Frame viewer available from LSAF 5.4	Available for .Rdata files and other
R version	4.4.0(Puppy Cup)	> 4.4.0

CONCLUSION

The Life Sciences Analytics Framework (LSAF) offers a secure work environment with features to enhance clinical research and development. R in LSAF will likely increase the use of R in statistical programming and statistics, in addition to SAS.

LSAF version 5.3 and later integrates with R, allowing users to write programs in SAS or R sessions for clinical data analysis. LSAF 5.4 includes data explorer support for R data frames, providing a more complete programming environment. The R session only offers code and log windows, while the SAS session includes dedicated windows for results, submission inputs, and submission outputs. The log window in the R session shows all results generated during R code execution for monitoring and debugging. Using R within LSAF offers several key benefits, including compliance, performance, and traceability. The platform provides a base set of pre-installed R packages and does not permit direct downloading of packages from CRAN. Users can add any additional packages needed for their analysis by creating a local library or miniCRAN in the workspace or repository. LSAF currently supports R version 4.4.0 (Puppy Cup), whereas RStudio has advanced to version > 4.4.0. Keeping R versions in LSAF up-to-date would help resolve compatibility issues with the latest R packages.

REFERENCES

- SAS Institute Inc. 2021. SAS® Life Science Analytics Framework 5.4: User's Guide. Cary, NC: SAS Institute Inc.
- Kuan, William YuanHao, and Bouchard, Olivier. 2023. "Driving Better Health for More People with SAS Life Science Analytics Framework (SAS LSAF)." Proceedings of the PHUSE 2023 Conference. Available at https://www.lexjansen.com/phuse/2023/sd/PAP_SD02.pdf
- Vervuren, Paul, and Ferran, Jean-Marc. 2021. "R and SAS Working Together: DNR's Experience Moving to LSAF 5.3." Proceedings of the PHUSE 2021 Conference. Available at https://www.lexjansen.com/phuse/2021/tt/PRE_TT07.pdf
- Hendrickx, John. 2021. "R and LSAF – Challenges and Solutions." Proceedings of the PHUSE 2021 SDE EMEA Autumn Conference. Available at https://phuse.s3.eu-central-1.amazonaws.com/Archive/2021/SDE/EU/Virtual%20-%20Autumn/PRE_EMEA26.pdf
- Becker, Matt, and Desai, Pritesh. 2025. "Leveraging R for Statistical Analysis in LSAF Q&A, Slides, and On-Demand Recording." SAS Ask the Expert Webinar, January 13, 2025. Available at <https://communities.sas.com/t5/Ask-the-Expert/Leveraging-R-for-Statistical-Analysis-in-LSAF-Q-amp-A-Slides-and/ta-p/955967>

ACKNOWLEDGMENTS

I would like to thank my managers, Kevin Najarian and Joe Luckanish, at Philips and ICON plc, for their guidance and support during the preparation of this paper.

CONTACT INFORMATION

Your comments and questions are valued and encouraged.

Contact the author at:

Author Name: Praneeth Adidela

Company: ICON plc

Email: praneeth.adidela@iconplc.com
praneethadidela@gmail.com

Any brand and product names are trademarks of their respective companies.